UNIVERSIDADE DE MOGI DAS CRUZES
SISTEMAS DE INFORMAÇÃO

DANIEL CARDOSO LEÃO

SISTEMA DE LOGIN SEGURO COM SPRING BOOT, THYMELEAF E MONGODB ATLAS:

documentação técnica

MOGI DAS CRUZES
2026

DANIEL CARDOSO LEÃO

SISTEMA DE LOGIN SEGURO COM SPRING BOOT, THYMELEAF E MONGODB ATLAS:

documentação técnica

Documentação técnica apresentada à disciplina de Aplicativos Web do curso de Sistemas de Informação da Universidade de Mogi das Cruzes, como requisito parcial para a composição da nota da avaliação M1.

Professor: Prof. Alessandro Aparecido da Silva Horas.

MOGI DAS CRUZES

2026

RESUMO

Este documento descreve o desenvolvimento de um sistema de login seguro construído com Java 21, Spring Boot 4, Spring Security 7, Thymeleaf e MongoDB Atlas. O sistema oferece cadastro, autenticação e encerramento de sessão de usuários, com senhas armazenadas como hash BCrypt, validação dos dados de entrada e controle de acesso baseado em três perfis hierárquicos: administrador, gerente e usuário. Uma única tela de gestão de usuários se adapta ao perfil: o gerente apenas consulta e o administrador altera perfis, ativa, desativa, desbloqueia e exclui contas. Os dados dos usuários e as sessões HTTP são persistidos em um cluster do MongoDB Atlas acessado por conexão criptografada com TLS, com as credenciais mantidas fora do código-fonte. A interface foi construída com Thymeleaf e o Thymeleaf Layout Dialect, separando estrutura, componentes visuais e tokens de tema, o que permite trocar a identidade visual por configuração, sem alterar a lógica da aplicação. São apresentadas a arquitetura em camadas, a integração com o banco de dados, os mecanismos de segurança adotados, as principais decisões de design e um guia para adaptar a base a novos projetos, em especial ao Projeto Final de Curso.

Palavras-chave: Spring Boot. Spring Security. MongoDB Atlas. Thymeleaf. Autenticação.

ABSTRACT

This document describes the development of a secure login system built with Java 21, Spring Boot 4, Spring Security 7, Thymeleaf and MongoDB Atlas. The system provides user sign-up, authentication and logout, with passwords stored as BCrypt hashes, input validation and access control based on three hierarchical roles: administrator, manager and user. A single user management screen adapts to the role: managers can only view it, while administrators change roles, enable, disable, unlock and delete accounts. User data and HTTP sessions are persisted in a MongoDB Atlas cluster through a TLS-encrypted connection, with credentials kept out of the source code. The interface was built with Thymeleaf and the Thymeleaf Layout Dialect, separating structure, visual components and theme tokens, so the visual identity can be changed through configuration without touching the application logic. The layered architecture, the database integration, the security mechanisms, the main design decisions and a guide for adapting the system to new projects are presented.

Keywords: Spring Boot. Spring Security. MongoDB Atlas. Thymeleaf. Authentication.

LISTA DE ILUSTRAÇÕES

Figura 1 - Arquitetura em camadas do Secure Login 12
Figura 2 - Tela de login 22
Figura 3 - Gestão de usuários vista pelo administrador 22
Figura 4 - Consulta de usuários vista pelo gerente 23
Figura 5 - Gestão de usuários com o tema dark 24

Quadro 1 - Tecnologias utilizadas no projeto 10
Quadro 2 - Pacotes do projeto (br.com.danielcardoso.securelogin) 13
Quadro 3 - Matriz de acesso por perfil 13
Quadro 4 - Medidas de segurança da conexão com o banco 15
Quadro 5 - Configuração da conexão no application.yml 16
Quadro 6 - Variáveis do arquivo de configuração (.env.example) 16
Quadro 7 - Exemplo de documento da coleção users 17
Quadro 8 - Configuração do codificador de senhas 18
Quadro 9 - Regras de autorização e hierarquia de perfis 19
Quadro 10 - Proteções implementadas contra ataques comuns 19
Quadro 11 - Trecho dos tokens de design do tema default 21
Quadro 12 - Decisões de design e justificativas 25
Quadro 13 - Pontos de adaptação do sistema 26
Quadro 14 - Testes automatizados 27

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
AWS	Amazon Web Services
CSP	Content Security Policy
CSRF	Cross-Site Request Forgery
CSS	Cascading Style Sheets
DTO	Data Transfer Object
HTML	HyperText Markup Language
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IP	Internet Protocol
JDK	Java Development Kit
JSON	JavaScript Object Notation
LTS	Long-Term Support
MVC	Model-View-Controller
OWASP	Open Worldwide Application Security Project
PFC	Projeto Final de Curso
TLS	Transport Layer Security
URI	Uniform Resource Identifier
XSS	Cross-Site Scripting

SUMÁRIO

1 INTRODUÇÃO 8
1.1 Objetivo geral 8
1.2 Objetivos específicos 8
1.3 Organização do documento 8
2 TECNOLOGIAS UTILIZADAS 10
2.1 Spring Boot e Spring Security 10
2.2 Thymeleaf 10
2.3 MongoDB Atlas 11
3 ARQUITETURA DO SISTEMA 12
3.1 Organização dos pacotes 13
3.2 Perfis, rotas e navegação 13
3.3 Versionamento com Gitflow 14
4 INTEGRAÇÃO COM O MONGODB ATLAS 15
4.1 Configuração do cluster 15
4.2 Conexão segura 15
4.3 Coleção users 16
4.4 Coleção sessions 17
5 SEGURANÇA 18
5.1 Armazenamento de senhas 18
5.2 Autenticação 18
5.3 Autorização e hierarquia de perfis 19
5.4 Proteções adicionais 19
5.5 Validação dos dados 20
6 INTERFACE E TEMAS 21
6.1 Camadas da apresentação 21
6.2 Telas do sistema 21

7 PRINCIPAIS DECISÕES DE DESIGN 25
8 GUIA DE ADAPTAÇÃO PARA NOVOS PROJETOS 26
8.1 Execução local 26
9 TESTES E VALIDAÇÃO 27
10 CONCLUSÃO 28
REFERÊNCIAS 29

1 INTRODUÇÃO

Praticamente todo sistema web precisa identificar quem o está utilizando e decidir o que cada pessoa pode fazer. Falhas nesse processo estão entre as causas mais comuns de incidentes de segurança, como o vazamento de senhas armazenadas sem proteção adequada e o acesso indevido a funções administrativas. Por isso, autenticação e autorização devem ser tratadas como parte central da arquitetura, e não como um detalhe a ser resolvido ao final do desenvolvimento.

Este trabalho apresenta o **Secure Login**, um sistema de login seguro desenvolvido com Java, Spring Boot, Thymeleaf e MongoDB Atlas. Além de atender aos requisitos da atividade, o sistema foi organizado de forma modular e genérica para servir de base a outros projetos, em especial ao Projeto Final de Curso (PFC), sem mudanças na lógica central.

1.1 Objetivo geral

Desenvolver um sistema web de autenticação e autorização seguro, com persistência em nuvem no MongoDB Atlas e interface preparada para receber temas visuais personalizados.

1.2 Objetivos específicos

- implementar cadastro, login e logout com hash de senhas e validação dos dados;
- controlar o acesso às rotas e páginas de acordo com três perfis de usuário;
- utilizar o Spring Security para gerenciar autenticação e autorização;
- armazenar usuários e sessões no MongoDB Atlas por meio de conexão segura;
- construir a interface com Thymeleaf, mantendo o design separado da lógica de negócio;
- permitir a troca de tema e de layout apenas por configuração;
- documentar a estrutura, a integração com o banco e as decisões de design.

1.3 Organização do documento

A seção 2 apresenta as tecnologias utilizadas. A seção 3 descreve a arquitetura, a organização do código e o fluxo de navegação por perfil. A seção 4 detalha a integração com o MongoDB Atlas. A seção 5 trata dos mecanismos de segurança e a seção 6 da interface e dos temas. A seção 7 reúne as principais decisões de design e a seção 8 explica como adaptar

o sistema a novos projetos. A seção 9 apresenta os testes realizados e a seção 10 traz as considerações finais.

A apresentação deste trabalho segue a NBR 14724 (ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS, 2011), e as referências foram elaboradas conforme a NBR 6023 (ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS, 2018).

2 TECNOLOGIAS UTILIZADAS

A escolha das tecnologias priorizou ferramentas maduras, bem documentadas e alinhadas ao conteúdo da disciplina. O Quadro 1 resume a pilha tecnológica do projeto.

Quadro 1 - Tecnologias utilizadas no projeto

Tecnologia	Versão	Papel no sistema
Java (Eclipse Temurin)	21 LTS	Linguagem e plataforma de execução
Spring Boot	4.0.8	Configuração automática, servidor embarcado e gestão de dependências
Spring Security	7.0	Autenticação, autorização, CSRF, sessões e cabeçalhos de segurança
Spring Data MongoDB	5.0	Mapeamento de documentos e repositórios
MongoDB Spring Session	4.0	Armazenamento das sessões HTTP no MongoDB
Thymeleaf	3.1	Motor de templates do lado do servidor
Thymeleaf Layout Dialect	3.4	Layouts reutilizáveis (decoração de páginas)
Thymeleaf Extras Spring Security	3.1	Exibição condicional de elementos conforme o perfil
Jakarta Bean Validation	3.1	Validação declarativa dos formulários
MongoDB Atlas	8.0 (M0)	Banco de dados em nuvem
Maven Wrapper	3.9	Build reproduzível sem instalação prévia do Maven
JUnit 5, Mockito e Spring Security Test	-	Testes automatizados

Fonte: Elaborado pelo autor (2026).

2.1 Spring Boot e Spring Security

O Spring Boot simplifica a criação de aplicações Spring ao oferecer configuração automática e um servidor web embarcado, de modo que a aplicação é executada como um programa Java comum (SPRING, 2026a). O Spring Security é o módulo padrão do ecossistema para segurança: ele intercepta as requisições por meio de uma cadeia de filtros e aplica as regras de autenticação e autorização antes que elas cheguem aos controllers (SPRING, 2026b).

2.2 Thymeleaf

O Thymeleaf é um motor de templates que gera HTML no servidor. Seus templates são arquivos HTML válidos, enriquecidos com atributos próprios, o que facilita o trabalho de quem cuida do design (THYMELEAF, 2026). O Layout Dialect acrescenta o conceito de layout: as páginas declaram qual layout as decora e preenchem apenas as áreas de conteúdo (ULTRAQ, 2026).

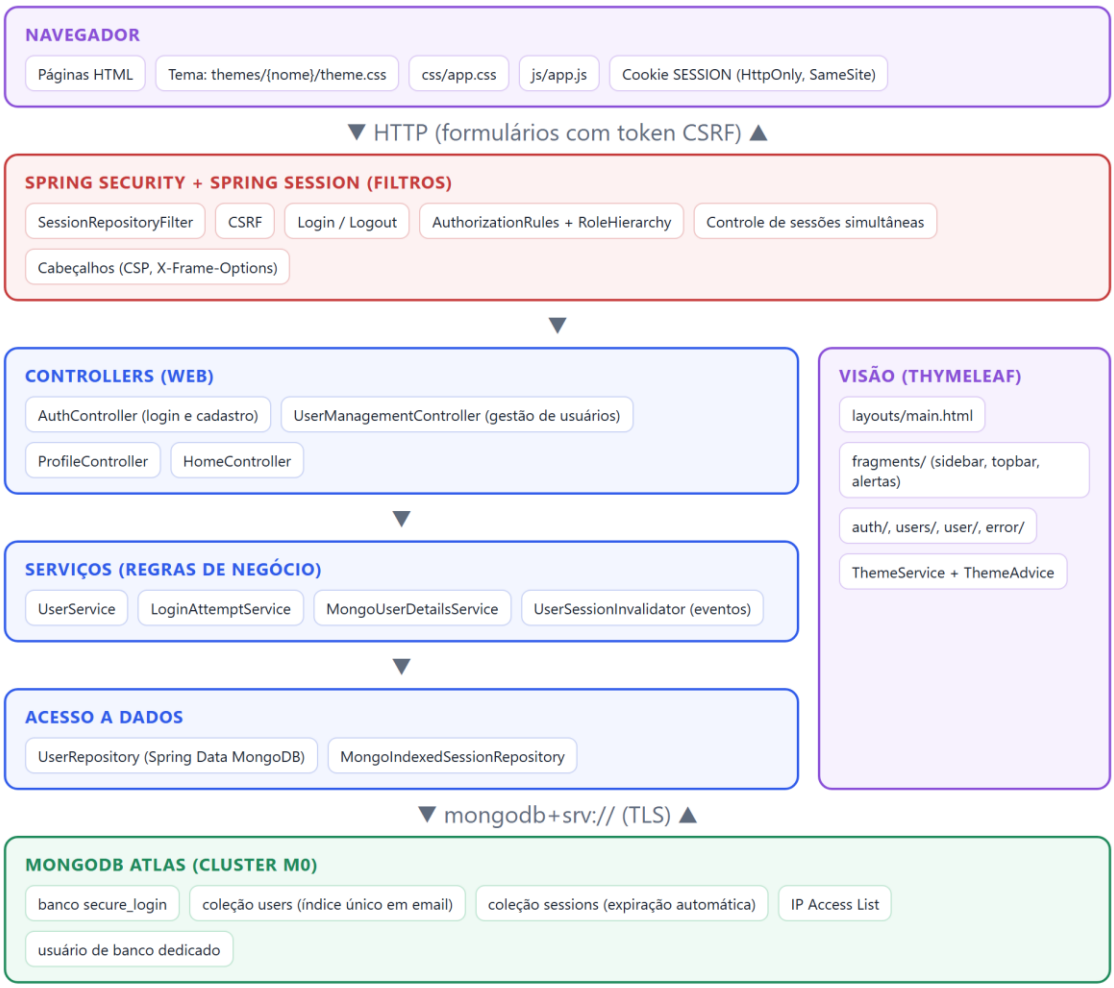
2.3 MongoDB Atlas

O MongoDB é um banco de dados orientado a documentos, que armazena registros em formato semelhante ao JSON. O Atlas é o serviço gerenciado do MongoDB na nuvem e oferece um plano gratuito (M0) adequado a projetos acadêmicos, com conexão criptografada, controle de acesso por usuário e lista de endereços IP autorizados (MONGODB, 2026).

3 ARQUITETURA DO SISTEMA

O sistema segue o padrão MVC em camadas. Cada camada conhece apenas a camada imediatamente abaixo, o que reduz o acoplamento e facilita a manutenção. A Figura 1 apresenta a visão geral da arquitetura.

Figura 1 - Arquitetura em camadas do Secure Login



Fonte: Elaborado pelo autor (2026).

Toda requisição passa primeiro pelos filtros do Spring Session, que recuperam a sessão no MongoDB, e do Spring Security, que verifica o token CSRF, a autenticação e o perfil exigido pela rota. Somente depois disso ela chega ao controller, que delega as regras de negócio aos serviços. Os serviços utilizam os repositórios para acessar o banco e devolvem os dados ao controller, que escolhe o template Thymeleaf responsável por gerar a página.

3.1 Organização dos pacotes

O código foi dividido em pacotes por responsabilidade, conforme o Quadro 2. Essa divisão permite localizar rapidamente onde cada ajuste deve ser feito e isola o que é específico deste trabalho do que é reaproveitável.

Quadro 2 - Pacotes do projeto (br.com.danielcardoso.securelogin)

Pacote	Responsabilidade	Principais classes
config	Configuração da aplicação, da segurança e da sessão	SecurityConfig, AuthorizationRules, MongoSessionConfig, AppProperties
security	Integração entre os usuários e o Spring Security	MongoUserDetailsService, LoginAttemptService, handlers de login, UserSessionInvalidator, AuthenticationUtils
user	Domínio de usuários: entidade, repositório, serviço, DTOs e telas da conta	User, Role, UserRepository, UserService, AuthController, ProfileController
admin	Gestão de usuários (consulta e administração)	UserManagementController
theme	Resolução do tema visual ativo	ThemeService, ThemeAdvice
web	Redirecionamento da raiz e dados comuns do layout	HomeController, CurrentUserAdvice
common	Validações e exceções reutilizáveis	StrongPassword, PasswordMatches, BusinessException, ResourceNotFoundException
bootstrap	Tarefas de inicialização	AdminSeeder

Fonte: Elaborado pelo autor (2026).

3.2 Perfis, rotas e navegação

O sistema possui três perfis organizados em hierarquia: o administrador herda as permissões do gerente, que herda as do usuário. O Quadro 3 mostra o que cada perfil pode acessar.

Quadro 3 - Matriz de acesso por perfil

Rota	Visitante	USER	MANAGER	ADMIN
/login, /register	Sim	-	-	-
/profile (Meu perfil)	Não	Sim	Sim	Sim
GET /users (consulta de usuários)	Não	Não	Sim	Sim
POST /users/** (alterar perfil, ativar, desativar, desbloquear, excluir)	Não	Não	Não	Sim

Fonte: Elaborado pelo autor (2026).

A gestão de usuários é uma única tela que se adapta ao perfil: o gerente vê a lista apenas para consulta, e o administrador vê a mesma lista com as ações. Após o login, gerente e administrador são levados diretamente a essa tela; o usuário comum, que não tem acesso a ela, é levado à página Meu perfil. Essa regra fica em um único método, usado pelo login, pela raiz do site e pelas telas de login e cadastro quando acessadas por quem já está autenticado.

Todo cadastro público recebe o perfil USER. O primeiro administrador é criado automaticamente na inicialização, a partir das variáveis de ambiente `ADMIN_EMAIL` e `ADMIN_PASSWORD`, e a partir daí pode promover outros usuários pela tela de gestão.

3.3 Versionamento com Gitflow

O código-fonte está publicado em https://github.com/danrerez/secure-login-spring e segue o fluxo Gitflow (ATLASSIAN, 2026). A branch `main` contém apenas versões estáveis, marcadas com tags; a branch `develop` integra o desenvolvimento; cada funcionalidade foi desenvolvida em uma branch `feature/*` própria, mesclada em `develop`; e a versão 1.0.0 foi preparada em uma branch `release/1.0.0` antes de ser incorporada à `main`.

4 INTEGRAÇÃO COM O MONGODB ATLAS

O MongoDB Atlas armazena duas coleções no banco `secure_login`: `users`, com os dados das contas, e `sessions`, com as sessões HTTP. Esta seção descreve como o banco foi configurado e como a aplicação se conecta a ele com segurança.

4.1 Configuração do cluster

A configuração no Atlas foi feita em cinco passos:

- criação de um projeto e de um cluster gratuito M0 na AWS, região São Paulo;
- criação de um usuário de banco exclusivo para a aplicação, com senha gerada automaticamente e permissão apenas de leitura e escrita;
- inclusão do endereço IP da máquina de desenvolvimento na IP Access List, de modo que conexões de outros endereços sejam recusadas;
- cópia da connection string no formato `mongodb+srv://`, disponível em Connect, Drivers;
- registro da connection string no arquivo `.env` local, que não é versionado.

O banco e as coleções não precisam ser criados manualmente: o MongoDB os cria na primeira gravação, e o Spring Data cria o índice único do campo e-mail na inicialização.

4.2 Conexão segura

A segurança da conexão foi tratada em várias frentes, resumidas no Quadro 4.

Quadro 4 - Medidas de segurança da conexão com o banco

Medida	Como foi aplicada
Criptografia em trânsito	A connection string `mongodb+srv://` habilita TLS automaticamente; os dados trafegam criptografados entre a aplicação e o Atlas.
Credenciais fora do código	A URI é lida da variável `MONGODB_URI`, definida no arquivo `.env` (ignorado pelo Git) ou no ambiente do servidor.
Restrição de rede	Somente endereços cadastrados na IP Access List conseguem se conectar ao cluster.
Menor privilégio	A aplicação usa um usuário de banco próprio, sem permissões administrativas sobre o cluster.
Arquivo de exemplo versionado	O `.env.example` documenta todas as variáveis necessárias, sem valores reais.

Fonte: Elaborado pelo autor (2026).

O Quadro 5 mostra o trecho do arquivo `application.yml` que configura a conexão. Os valores entre `${...}` são resolvidos a partir das variáveis de ambiente ou do arquivo `.env`, importado pela propriedade `spring.config.import`.

Quadro 5 - Configuração da conexão no application.yml

```
spring:
  config:
    import: optional:file:.env[.properties]
  mongodb:
    uri: ${MONGODB_URI}
    database: ${MONGODB_DATABASE:secure_login}
  data:
    mongodb:
      auto-index-creation: true
```

Fonte: Elaborado pelo autor (2026).

O Quadro 6 apresenta as variáveis do arquivo `.env.example`, entregue como modelo do arquivo de configuração; no arquivo, cada variável vem acompanhada de instruções de preenchimento. Para executar o sistema localmente, basta copiá-lo para `.env` e preencher os valores reais.

Quadro 6 - Variáveis do arquivo de configuração (.env.example)

```
MONGODB_URI=mongodb+srv://<usuario>:<senha>@<cluster>.xxxxx.mongodb.net/
            ?retryWrites=true&w=majority&tls=true&appName=<cluster>
MONGODB_DATABASE=secure_login
ADMIN_NAME=Administrador
ADMIN_EMAIL=admin@exemplo.com
ADMIN_PASSWORD=TroqueEssaSenha@123
APP_THEME=default
COOKIE_SECURE=false
THYMELEAF_CACHE=false
```

Fonte: Elaborado pelo autor (2026).

4.3 Coleção users

Cada usuário é um documento da classe `User`, mapeada com a anotação `@Document("users")`. O campo e-mail possui índice único, o que garante no próprio banco que não existam duas contas com o mesmo endereço, mesmo que duas requisições de cadastro cheguem ao mesmo tempo. O Quadro 7 mostra um exemplo de documento armazenado.

Quadro 7 - Exemplo de documento da coleção users

```
{
  "_id": "6ac7b77161375d84be01a195",
  "name": "Maria Silva",
  "email": "maria@exemplo.com",
  "password": "{bcrypt}$2a$12$Qd8f...",
  "role": "USER",
  "enabled": true,
  "failedLoginAttempts": 0,
  "lockedUntil": null,
  "lastLoginAt": "2026-10-08T15:40:12Z",
  "createdAt": "2026-10-08T15:31:57Z",
  "updatedAt": "2026-10-08T15:40:12Z"
}
```

Fonte: Elaborado pelo autor (2026).

Os campos `createdAt` e `updatedAt` são preenchidos automaticamente pelo recurso de auditoria do Spring Data (`@EnableMongoAuditing`). O acesso é feito pela interface `UserRepository`, que estende `MongoRepository`; consultas como `findByEmail` e `countByRole` são geradas automaticamente a partir do nome do método (SPRING, 2026c).

4.4 Coleção sessions

Por padrão, o servidor guarda as sessões HTTP na memória, e elas se perdem quando a aplicação é reiniciada. Com a anotação `@EnableMongoHttpSession`, as sessões passam a ser gravadas na coleção `sessions` do Atlas (SPRING, 2026d). Essa decisão traz três benefícios:

- o usuário continua logado mesmo após um reinício do servidor;
- várias instâncias da aplicação podem compartilhar as mesmas sessões, o que permite escalar horizontalmente;
- a aplicação consegue localizar e encerrar todas as sessões de um usuário, recurso usado quando o administrador altera o perfil, desativa ou exclui uma conta.

O tempo de expiração da sessão é configurável (`app.security.session-timeout`, padrão de 30 minutos) e o MongoDB remove automaticamente os documentos expirados. O cookie de sessão é marcado como `HttpOnly` e `SameSite=Lax`, e pode ser marcado como `Secure` em produção, quando a aplicação roda sob HTTPS.

5 SEGURANÇA

Esta seção descreve como o sistema protege as senhas, autentica os usuários, controla o acesso e se defende de ataques comuns em aplicações web.

5.1 Armazenamento de senhas

As senhas nunca são armazenadas em texto puro. Elas passam pelo algoritmo BCrypt, que gera um hash com salt aleatório e custo computacional ajustável (PROVOS; MAZIÈRES, 1999). Foi utilizado custo 12, valor que torna ataques de força bruta caros sem prejudicar a experiência do usuário, em linha com as recomendações da OWASP (OWASP FOUNDATION, 2026a). O codificador é um `DelegatingPasswordEncoder`, que grava o prefixo `{bcrypt}` no hash. Assim, o algoritmo pode ser trocado no futuro sem invalidar as senhas já cadastradas.

Quadro 8 - Configuração do codificador de senhas

```
@Bean
PasswordEncoder passwordEncoder() {
    return new DelegatingPasswordEncoder("bcrypt",
            Map.of("bcrypt", new BCryptPasswordEncoder(12)));
}
```

Fonte: Elaborado pelo autor (2026).

5.2 Autenticação

O login usa o formulário próprio do sistema, processado pelo Spring Security. A classe `MongoUserDetailsService` busca o usuário pelo e-mail e o converte em um `AuthenticatedUser`, que o Spring Security compara com a senha digitada. Após o login, o identificador da sessão é trocado para evitar ataques de fixação de sessão, e o hash da senha é apagado do objeto mantido em memória.

Para dificultar ataques de força bruta, a classe `LoginAttemptService` conta as tentativas incorretas e bloqueia a conta por 15 minutos após cinco erros seguidos. Todas as falhas de login exibem a mesma mensagem genérica, para que um atacante não consiga descobrir quais e-mails estão cadastrados (OWASP FOUNDATION, 2026b). O administrador pode desbloquear uma conta manualmente. Quando o próprio usuário troca a senha, as sessões abertas em outros dispositivos são encerradas, para que uma sessão obtida com a senha antiga deixe de valer.

5.3 Autorização e hierarquia de perfis

As regras de acesso por rota estão concentradas na classe `AuthorizationRules`, apresentada no Quadro 9. A hierarquia de perfis, declarada como um bean `RoleHierarchy`, faz com que o administrador seja aceito em todas as rotas do gerente, e o gerente em todas as rotas do usuário, sem repetição de regras.

Quadro 9 - Regras de autorização e hierarquia de perfis

```
auth.requestMatchers(PUBLIC_PATHS).permitAll()
    .requestMatchers(HttpMethod.POST, "/users/**").hasRole("ADMIN")
    .requestMatchers("/users/**").hasRole("MANAGER")
    .anyRequest().authenticated();

RoleHierarchyImpl.withDefaultRolePrefix()
    .role("ADMIN").implies("MANAGER")
    .role("MANAGER").implies("USER")
    .build();
```

Fonte: Elaborado pelo autor (2026).

Como defesa em profundidade, o controller de gestão de usuários também é protegido por `@PreAuthorize`: a classe exige o perfil MANAGER e cada método de alteração exige ADMIN. Assim, mesmo que uma regra de rota seja alterada por engano, as ações continuam protegidas. Nas telas, o dialeto `sec:` do Thymeleaf exibe apenas os menus permitidos, e os botões de ação só são gerados para o administrador.

5.4 Proteções adicionais

Quadro 10 - Proteções implementadas contra ataques comuns

Ameaça	Proteção adotada
Falsificação de requisições (CSRF)	Token CSRF obrigatório em todos os formulários; o logout só é aceito via POST.
Força bruta	Bloqueio temporário após 5 tentativas incorretas e custo elevado do BCrypt.
Enumeração de usuários no login	Mensagem de erro única para qualquer falha de login.
Senha comprometida	Ao trocar a senha, as sessões abertas em outros dispositivos são encerradas.
Fixação e roubo de sessão	Troca do identificador no login, cookie HttpOnly e SameSite, limite de 3 sessões simultâneas por usuário.
Privilégios desatualizados	Sessões do usuário encerradas quando o perfil é alterado ou a conta é desativada ou excluída.
Cross-site scripting (XSS)	Escape automático do Thymeleaf e Content Security Policy sem scripts inline.
Clickjacking	Cabeçalho X-Frame-Options: DENY e diretiva frame-ancestors 'none'.
Perda do único administrador	O administrador não pode alterar o próprio perfil, desativar ou excluir a própria conta.

Fonte: Elaborado pelo autor (2026).

5.5 Validação dos dados

Os formulários são validados no servidor com Jakarta Bean Validation. Além das anotações padrão (`@NotBlank`, `@Email`, `@Size`), foram criadas duas validações reutilizáveis: `@StrongPassword`, que exige de 8 a 64 caracteres com letra maiúscula, minúscula, número e símbolo, respeitando o limite de 72 bytes do BCrypt; e `@PasswordMatches`, que confere se a senha e a confirmação são iguais. O e-mail é normalizado (sem espaços e em letras minúsculas) antes de ser gravado, e a unicidade é garantida pelo serviço e pelo índice do banco.

6 INTERFACE E TEMAS

A interface foi construída para que o visual possa ser alterado sem tocar na lógica de negócio. Os templates não contêm regras: recebem dos controllers apenas os dados que exibem, por meio de objetos de leitura como `UserSummary`, que nunca expõe o hash da senha.

6.1 Camadas da apresentação

A apresentação foi dividida em três camadas independentes:

- **layout** (`templates/layouts/main.html`): estrutura comum a todas as páginas. Para o usuário logado, monta a barra lateral de navegação e a barra superior com o menu do usuário (Meu perfil e Sair); nas telas públicas de login e cadastro, exibe apenas o formulário centralizado. Cada página preenche só o bloco `content`;
- **componentes** (`static/css/app.css`): barra lateral, botões, cartões, tabelas e formulários escritos somente com variáveis CSS, sem nenhuma cor fixa;
- **tema** (`static/themes/<nome>/theme.css`): define os valores das variáveis, como cores, fontes, raios de borda e sombras.

O tema e o layout ativos são definidos pelas propriedades `app.theme.name` e `app.theme.layout`, que podem ser alteradas pelas variáveis `APP_THEME` e `APP_LAYOUT`. A classe `ThemeService` verifica se o tema existe e, caso não exista, registra um aviso e volta para o tema padrão.

Quadro 11 - Trecho dos tokens de design do tema default

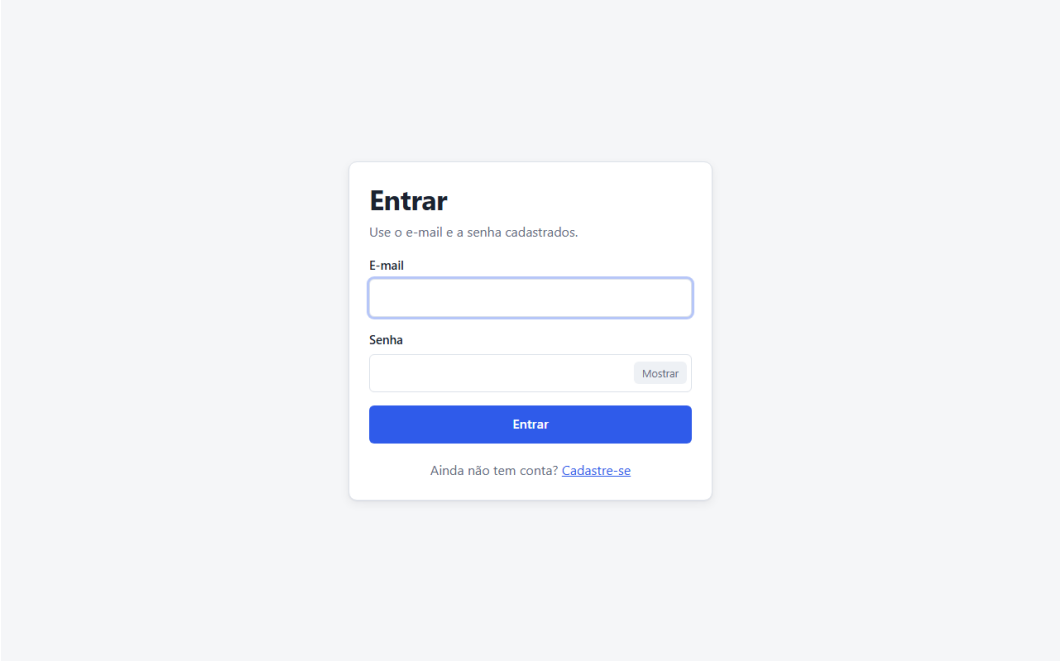
```
:root {
    --font-sans: "Segoe UI", system-ui, Arial, sans-serif;
    --color-bg: #f5f6f8;
    --color-surface: #ffffff;
    --color-text: #1b2230;
    --color-primary: #2f5bea;
    --sidebar-bg: #ffffff;
    --radius: 10px;
}
```

Fonte: Elaborado pelo autor (2026).

6.2 Telas do sistema

O sistema tem poucas telas, de propósito: login, cadastro, gestão de usuários e Meu perfil. A Figura 2 mostra a tela de login, centralizada e sem distrações. A tela de cadastro segue o mesmo padrão e exibe as mensagens de validação abaixo de cada campo.

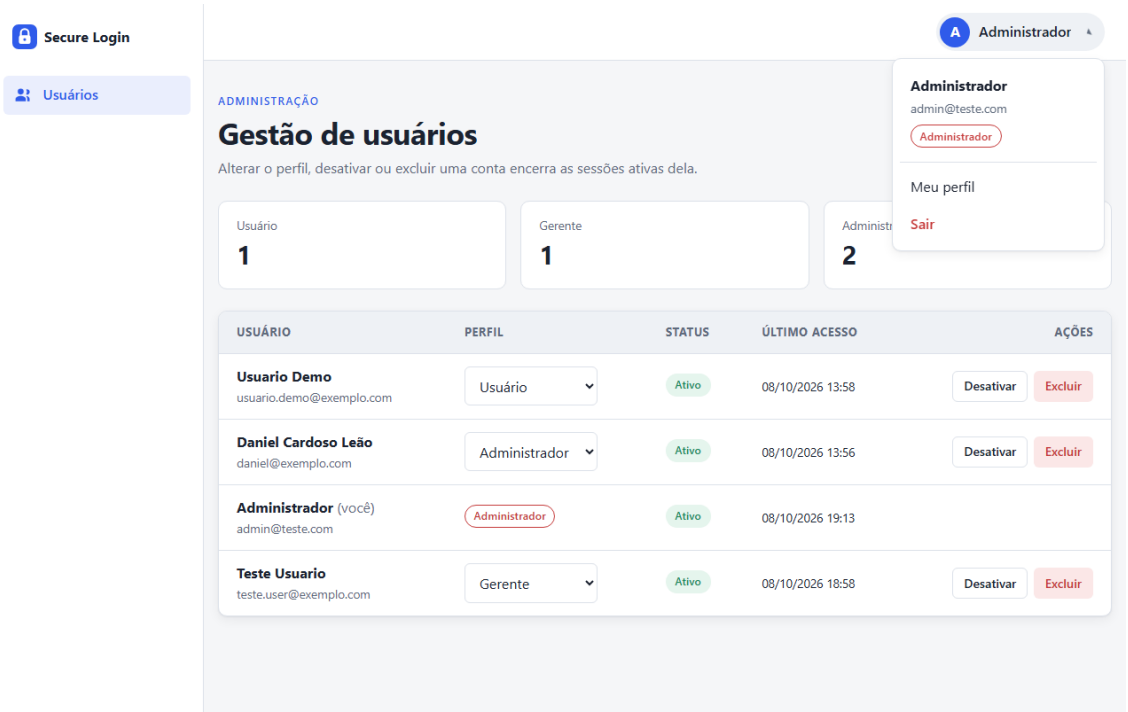
Figura 2 - Tela de login



Fonte: Elaborado pelo autor (2026).

A Figura 3 mostra a gestão de usuários vista pelo administrador, com o perfil editável na própria tabela, os botões de ação e o menu do usuário aberto no canto superior direito. Na linha do próprio administrador não aparecem ações, para que ele não remova o próprio acesso.

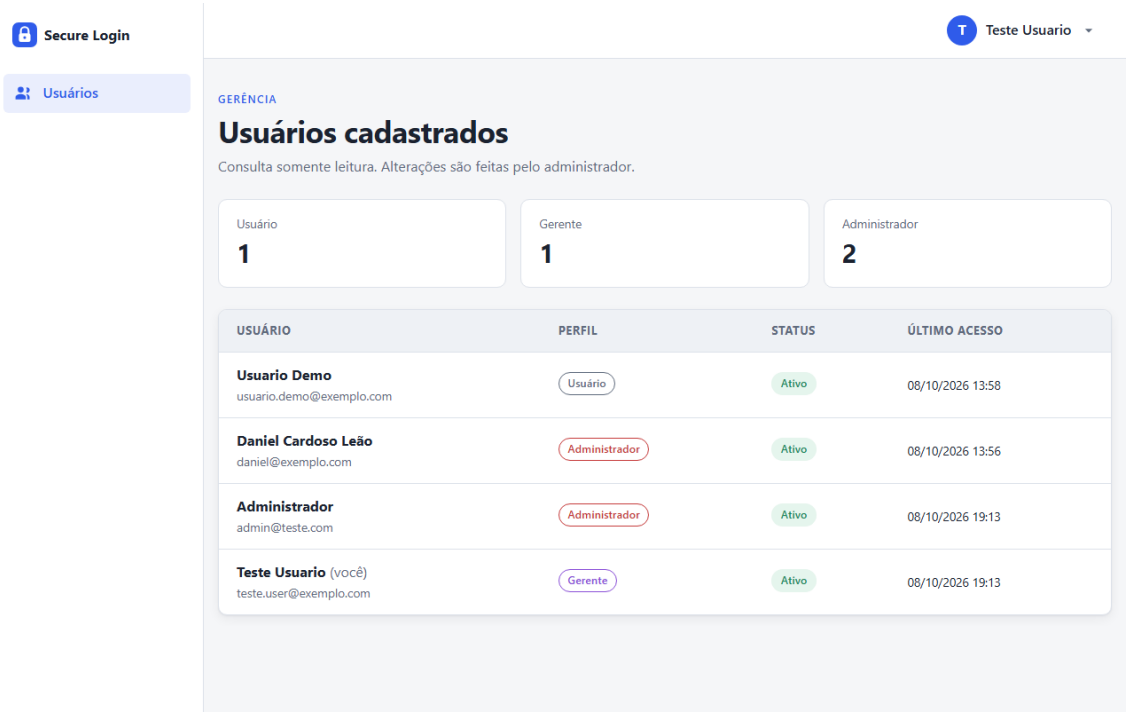
Figura 3 - Gestão de usuários vista pelo administrador



Fonte: Elaborado pelo autor (2026).

A Figura 4 mostra a mesma tela acessada por um gerente: a lista e os indicadores são os mesmos, mas o perfil aparece apenas como etiqueta e a coluna de ações não é exibida.

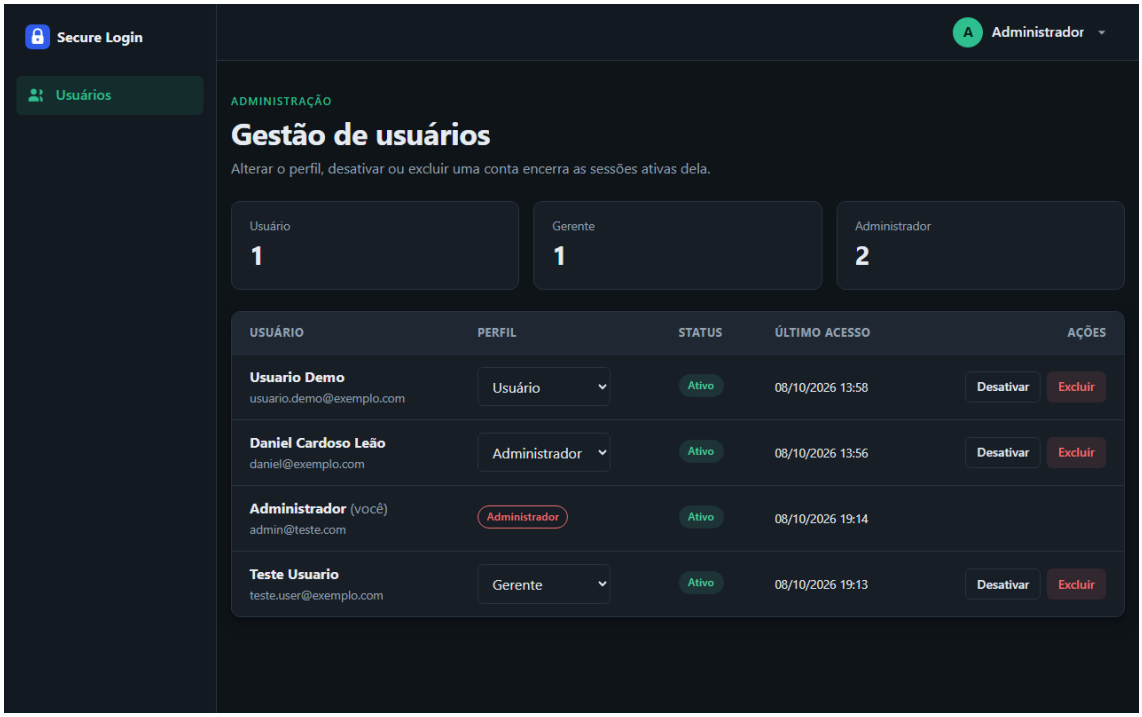
Figura 4 - Consulta de usuários vista pelo gerente



Fonte: Elaborado pelo autor (2026).

A Figura 5 mostra a tela da Figura 3 com o tema escuro, ativado apenas com `APP_THEME=dark`, sem nenhuma alteração no código. O layout também é responsivo: em telas pequenas, a barra lateral vira um menu que abre por um botão.

Figura 5 - Gestão de usuários com o tema dark



Fonte: Elaborado pelo autor (2026).

7 PRINCIPAIS DECISÕES DE DESIGN

O Quadro 12 reúne as decisões mais relevantes tomadas durante o desenvolvimento e suas justificativas.

Quadro 12 - Decisões de design e justificativas

Decisão	Justificativa
Java 21 e Spring Boot 4.0	Java 21 é a versão LTS atual da plataforma. A linha 4.0 do Spring Boot foi escolhida por ser a mais recente compatível com o módulo de sessões no MongoDB.
Sessões no MongoDB (Spring Session)	Atende ao requisito de armazenar sessões no Atlas, mantém o login após reinícios e permite encerrar as sessões de um usuário.
BCrypt custo 12 com DelegatingPasswordEncoder	Algoritmo recomendado para senhas e possibilidade de migrar de algoritmo sem invalidar contas.
Um perfil por usuário com hierarquia	Modelo simples de entender e de exibir na tela; a hierarquia evita duplicar regras de acesso.
Tela única de gestão de usuários	Gerente e administrador usam a mesma tela; muda apenas o que cada um pode fazer, o que reduz código duplicado e deixa a diferença de perfil evidente.
Tela inicial por perfil	Cada perfil entra direto onde trabalha, sem uma página intermediária; a regra fica em um único método.
Regras de acesso centralizadas em AuthorizationRules	Um único lugar mostra quem acessa o quê, o que facilita auditoria e adaptação.
Defesa em profundidade com @PreAuthorize	Protege os métodos mesmo que uma regra de rota seja alterada por engano.
Mensagem de erro de login genérica	Evita que um atacante descubra quais e-mails estão cadastrados.
Eventos para encerrar sessões	O UserService publica um evento e a camada de segurança reage, sem que o serviço dependa do Spring Session.
DTOs para formulários e leitura	Separam o que a tela recebe do documento salvo e impedem que o hash da senha chegue às páginas.
Configuração por variáveis de ambiente	Nenhuma credencial no repositório; o mesmo código roda em qualquer ambiente.
Temas por variáveis CSS e layout configurável	Trocar a identidade visual exige apenas um novo arquivo de tema, sem mexer em templates ou Java.
Sem bibliotecas CSS ou JavaScript externas	Permite uma Content Security Policy restritiva e elimina dependência de CDNs.
Administrador inicial por variável de ambiente	O cadastro público nunca cria administradores; o primeiro é definido por quem implanta o sistema.

Fonte: Elaborado pelo autor (2026).

8 GUIA DE ADAPTAÇÃO PARA NOVOS PROJETOS

O sistema foi pensado para ser a base de autenticação de outros projetos, como o PFC. O Quadro 13 indica onde fazer cada tipo de adaptação.

Quadro 13 - Pontos de adaptação do sistema

Necessidade	Onde alterar
Renomear ou adicionar perfis	Enum `Role` e método `roleHierarchy()` em `SecurityConfig`
Proteger uma nova área do sistema	Adicionar o prefixo da rota e o perfil mínimo em `AuthorizationRules`
Adicionar itens ao menu	`templates/fragments/sidebar.html`, protegendo cada item com `sec:authorize`
Mudar a tela inicial de um perfil	Método `homeFor` em `AuthenticationUtils`
Adicionar campos ao usuário	`User`, `RegisterForm`, `UserSummary` e os templates de cadastro e perfil
Mudar a identidade visual	Criar `static/themes/<nome>/theme.css` e definir `APP_THEME=<nome>`
Mudar a estrutura das páginas	Criar um novo arquivo em `templates/layouts/` e definir `APP_LAYOUT`
Ajustar a política de bloqueio ou de sessão	Propriedades `app.security.*` no `application.yml`
Usar outro banco no Atlas	Variáveis `MONGODB_URI` e `MONGODB_DATABASE`

Fonte: Elaborado pelo autor (2026).

Como exemplo, em um PFC de gestão de clínicas, bastaria trocar os perfis por administrador, recepcionista e paciente, proteger a nova área de agenda em `AuthorizationRules`, incluir o item Agenda na barra lateral e criar um tema com as cores do projeto. Cadastro, login, bloqueio por tentativas, sessões no Atlas e gestão de usuários continuariam funcionando sem alteração.

8.1 Execução local

Para executar o sistema localmente são necessários o JDK 21 e uma conta no MongoDB Atlas. Os passos são:

- clonar o repositório https://github.com/danrerez/secure-login-spring;
- copiar o arquivo `.env.example` para `.env` e preencher a connection string e os dados do administrador;
- executar `./mvnw spring-boot:run` (ou `mvnw.cmd spring-boot:run` no Windows);
- acessar http://localhost:8080 e entrar com o e-mail e a senha do administrador.

O arquivo README do repositório traz o passo a passo completo, inclusive a criação do cluster no Atlas.

9 TESTES E VALIDAÇÃO

O projeto possui 31 testes automatizados, executados com `./mvnw test`. Eles não dependem do Atlas: os testes da camada web usam `@WebMvcTest` com a configuração real do Spring Security e o serviço de usuários simulado com Mockito. O Quadro 14 resume o que é verificado.

Quadro 14 - Testes automatizados

Classe de teste	O que verifica
AccessControlTest (16 testes)	Acesso de visitante, USER, MANAGER e ADMIN a cada rota; tela inicial de cada perfil; visão somente leitura do gerente e ações do administrador na mesma tela; bloqueio das ações para gerente e usuário; resposta 404; encerramento das outras sessões na troca de senha; CSRF; cabeçalhos de segurança; validação e sucesso do cadastro.
UserServiceTest (5 testes)	Hash BCrypt e normalização do e-mail no cadastro; e-mail duplicado; troca de senha exige a senha atual; administrador não altera o próprio perfil; alteração de perfil encerra sessões.
LoginAttemptServiceTest (3 testes)	Bloqueio após 5 tentativas; reset do contador após login correto; nova contagem após o fim do bloqueio.
PasswordValidationTest (7 testes)	Aceitação de senha forte; rejeição de senhas fracas; confirmação divergente.

Fonte: Elaborado pelo autor (2026).

Além dos testes automatizados, o sistema foi validado manualmente com o banco no Atlas. Foram confirmados o cadastro com a senha gravada como hash BCrypt, o login de cada perfil com o redirecionamento para a tela inicial correta, o bloqueio com status 403 das áreas não permitidas, o registro da sessão na coleção `sessions`, o logout, o bloqueio da conta após cinco senhas incorretas, a promoção de um usuário a gerente com encerramento imediato da sessão antiga e a troca para o tema escuro apenas por configuração.

10 CONCLUSÃO

O Secure Login atende aos requisitos propostos: oferece cadastro, login e logout seguros, com senhas protegidas por BCrypt e validação dos dados; controla o acesso por três perfis hierárquicos com o Spring Security; armazena usuários e sessões no MongoDB Atlas por meio de conexão criptografada e credenciais fora do código; e apresenta uma interface em Thymeleaf na qual layout, componentes e tema são camadas independentes.

Além dos requisitos mínimos, foram incorporadas práticas recomendadas de segurança, como o bloqueio por tentativas, o encerramento de sessões quando os privilégios ou a senha mudam, a Content Security Policy e a proteção contra enumeração de usuários no login. A organização em pacotes por responsabilidade, a centralização das regras de acesso e a configuração por variáveis de ambiente tornam o sistema uma base reaproveitável para o Projeto Final de Curso.

Como limitação, o bloqueio por tentativas pode ser usado de propósito para bloquear uma conta conhecida por alguns minutos, e o cadastro informa quando um e-mail já está em uso. Como trabalhos futuros, podem ser adicionados um CAPTCHA ou limite por endereço IP no login, a recuperação de senha por e-mail, a autenticação em dois fatores, o login com provedores externos (OAuth 2.0) e um registro de auditoria das ações administrativas.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6023**: informação e documentação: referências: elaboração. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2011.

ATLASSIAN. **Gitflow workflow**. 2026. Disponível em: https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow. Acesso em: 8 out. 2026.

MONGODB. **MongoDB Atlas documentation**. 2026. Disponível em: https://www.mongodb.com/docs/atlas/. Acesso em: 8 out. 2026.

OWASP FOUNDATION. **Password storage cheat sheet**. 2026a. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html. Acesso em: 8 out. 2026.

OWASP FOUNDATION. **Authentication cheat sheet**. 2026b. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html. Acesso em: 8 out. 2026.

PROVOS, Niels; MAZIÈRES, David. A future-adaptable password scheme. In: USENIX ANNUAL TECHNICAL CONFERENCE, 1999, Monterey. **Proceedings** [...]. Berkeley: USENIX Association, 1999.

SPRING. **Spring Boot reference documentation**. 2026a. Disponível em: https://docs.spring.io/spring-boot/index.html. Acesso em: 8 out. 2026.

SPRING. **Spring Security reference**. 2026b. Disponível em: https://docs.spring.io/spring-security/reference/index.html. Acesso em: 8 out. 2026.

SPRING. **Spring Data MongoDB reference documentation**. 2026c. Disponível em: https://docs.spring.io/spring-data/mongodb/reference/. Acesso em: 8 out. 2026.

SPRING. **Spring Session reference**. 2026d. Disponível em: https://docs.spring.io/spring-session/reference/. Acesso em: 8 out. 2026.

THYMELEAF. **Tutorial: using Thymeleaf**. 2026. Disponível em: https://www.thymeleaf.org/doc/tutorials/3.1/usingthymeleaf.html. Acesso em: 8 out. 2026.

ULTRAQ. **Thymeleaf Layout Dialect**. 2026. Disponível em: https://ultraq.github.io/thymeleaf-layout-dialect/. Acesso em: 8 out. 2026.